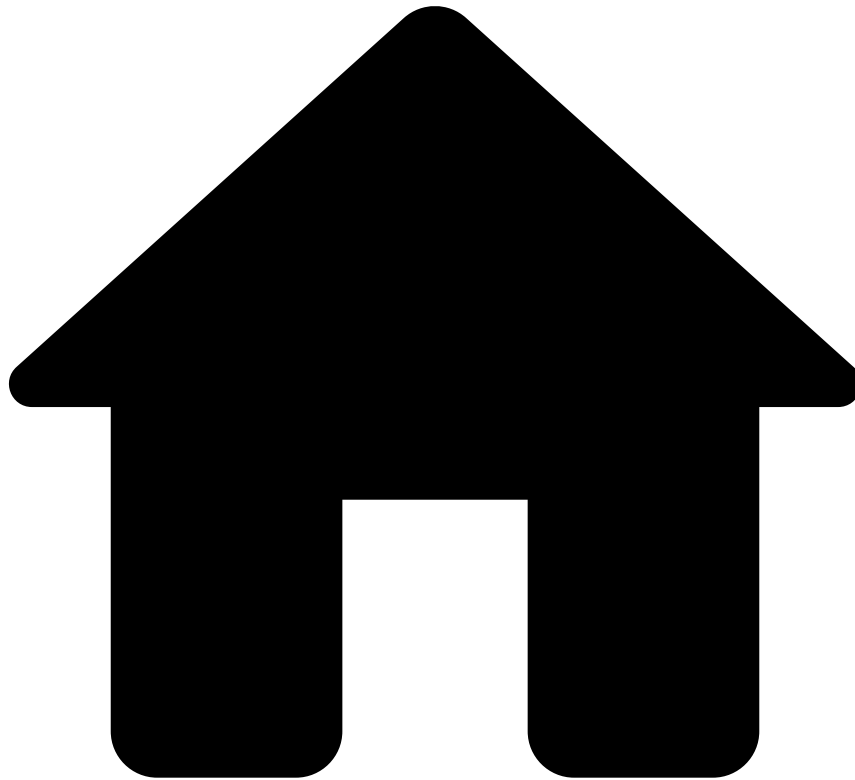


•



- Best Practices
- [AutoML](#)
- Hyperparameter Tuning Stage

On this page

Was this page helpful? 

Best Practices for AutoML's Hyperparameter Tuning Stage

The last stage of AutoML is hyperparameter tuning (HPT). Here, AutoML will train a range of models through OpenML, tuning their hyperparameters in one go. There are some important considerations during this stage:

1. This will use the **model training** logic: 1 big Spark task per model.
2. Trained models are kept on the machine where they are trained, and then discarded. This happens because the goal of HPT is getting the best model configuration and not all the model binaries. At the end of the run the best model will be retrained and kept within Pulse.
3. Evaluations will **not be distributed**. They run with as much parallelism as the model training, i.e., 1 Spark task.

4. If 1 model fails to train, then the HPT stage also fails. This happens because HPT considered a single job as part of the AutoML pipeline. If you want to re-run HPT under a different job configuration, you can do so by using an empty semantic file, and selecting the appropriate data source as input, i.e., the one with features already computed.
5. It will use **Random Search**.

That being said, most best practices here rely on configuration. Check the [model training documentation](#) that explains how to create a job profile for model training. From there you will see that you should:

- Size the executor for the biggest machines in the cluster.
- Allocate all the cores of the machine for the single task with `spark.task.cpus`.
- Allocate all the cores and memory of the machine with `spark.executor.cores` and `spark.executor.memory`. Leave only a little overhead with `spark.executor.memoryOverhead` to account for serialization/deserialization overheads.
- No need for any meaningful YARN memory overhead since there are no shuffles.
- Reduce `spark.memory.fraction` significantly, e.g., to 0.05; since there is no shuffling or caching, it is all user memory for the training.

Remember that one of the possible sources of failure is that the data must fit in memory during training. Because of this you should tune your model training jobs to use the largest machines. The plural here is important. Be aware that if you tune your job to fit only a few machines (say, 1 or 2), you will only be able to run that same number of models in parallel. In addition, to speed up this job, try leveraging the CPU cores the machines have available. Don't forget that the job will not be distributed. It will run 1 task per model.

This approach contrasts with the typical Model evaluations. In this case your evaluation will be done streaming your validation data in one very long task. That being said, **if your data set is large, use a small time period for the evaluations in HPT.**

Hyperparameter tuning algorithm

The algorithm used in AutoML is different from the one used on a regular HPT job.

The manual HPT job uses grid search, i.e., the user specifies which parameters will be used. AutoML uses Random Search instead. For random search, Pulse will draw the possible parameters from the following probability functions:

Feedzai Random Forest

Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
Depth	20	5	100	x
Trees	100	50	500	x
In-bag factor	0.65	0.2	1	x
N. Features	$0.3 \times k$	-4	-0.5	$k \times 2^x$, where k is the number of features.
Split method	one-vs-others	one-vs-others all-vs-all		Discrete.
Min. instances per split	2	0	1/3	n^x , where n is the number of samples.
Prior	Parent			No search.

Generalized Linear Model (GLM)

The lambda search is deferred to H2O to perform it internally.

Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
alpha	empty	0	1	x
lambda_search	On			

XGBoost

Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
ntrees	100	50	500	x
learn_rate	0.2	-6	-1	2^x
max_depth	6	2	15	x
min_child_weight	2	0	5	2^x
sample_rate	1	0	1	x
col_sample_rate	0.5	0	1	x
col_sample_rate_per_tree	1	0	1	x
reg_alpha	1	-5	3	2^x
reg_lambda	1	-5	3	2^x

Gradient Boosting Machine (GBM)

Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
ntrees	100	50	500	x
learn_rate	0.1	-6	-1	2^x
max_depth	6	2	15	x
min_rows	2	0	5	2^x
sample_rate	1	0	1	x
col_sample_rate	0.5	0	1	x

Parameter	Default	$x_{\min}$	$x_{\max}$	$p(x)$
col_sample_rate_per_tree	1	0	1	x